

Software Requirements Specification (SRS)

WhatsApp-Like Messaging System

Version 1.0

Table of Contents

1. Introduction
 2. Overall Description
 3. System Features
 4. External Interface Requirements
 5. Non-Functional Requirements
 6. Class Specifications
 7. Implementation Guidelines
-

1. Introduction

1.1 Purpose

This Software Requirements Specification document provides a detailed overview of the WhatsApp-like messaging system for software engineer trainees. The document describes the project's features, functionality, and implementation requirements to ensure a standardized approach to completion.

1.2 Scope

The system is a console-based messaging application that simulates core WhatsApp functionalities including user management, private chats, group chats, and message handling. Trainees will implement the missing functionality based on this specification. All data will be stored in memory during runtime only.

1.3 Definitions and Acronyms

- SRS: Software Requirements Specification
- User: A person using the application
- Chat: A conversation between two or more users

- Message: A unit of communication sent in a chat
 - Timestamp: Date and time when an action occurred
-

2. Overall Description

2.1 Product Perspective

This is a standalone console application written in C++ that simulates a messaging platform. It serves as a learning tool for understanding object-oriented programming concepts, class relationships, and basic data management. All data exists only in memory during runtime and is lost when the application closes.

2.2 User Characteristics

- Trainee Developers: Will complete the implementation
- End Users: People using the application for messaging during a single session

2.3 Operating Environment

- Windows/Linux/macOS operating systems
- C++ compiler supporting C++11 or later
- Console/terminal interface

2.4 Design and Implementation Constraints

- All classes must be implemented as specified
 - No external libraries except standard C++ libraries
 - Console-based interface only
 - All data stored in memory (no file persistence)
-

3. System Features

3.1 User Management

Description: Users can create accounts, log in, and manage their profiles.

Functional Requirements:

1. FR1: Users must register with username, password, and phone number

2. FR2: Usernames must be unique in the system
3. FR3: Passwords must be at least 6 characters long
4. FR4: Users can update their status and phone number
5. FR5: Last seen timestamp updates when user logs out or performs actions
6. FR6: Users can change their password

3.2 Messaging

Description: Users can send and manage messages in chats.

Functional Requirements:

1. FR7: Messages must contain sender, content, and timestamp
2. FR8: Messages can be sent in private or group chats
3. FR9: Users can delete their own messages
4. FR10: Messages have status (sent, delivered, read)
5. FR11: Users can reply to specific messages
6. FR12: Users can search messages by keyword
7. FR13: Users can add emojis (represented as text codes like ":)") or ":(") to messages

3.3 Private Chats

Description: One-on-one conversations between two users.

Functional Requirements:

1. FR14: Private chat created between exactly two users
2. FR15: Show typing indicator when user is typing
3. FR16: Display chat with messages in chronological order

3.4 Group Chats

Description: Conversations with multiple participants.

Functional Requirements:

1. FR17: Groups require a name and at least 2 participants
2. FR18: Creator becomes admin automatically

3. FR19: Admins can add other admins
 4. FR20: Admins can remove participants
 5. FR21: Groups can have descriptions
 6. FR22: Users can request to join groups
 7. FR23: Display all participants and admins when viewing group
-

4. External Interface Requirements

4.1 User Interfaces

- Console-based text interface
- Menu-driven navigation with numbered options
- Clear prompts for user input
- Formatted display of messages and chats

4.2 Software Interfaces

- Standard input/output streams
- Time/date functions from <ctime> library

4.3 Communication Interfaces

- No network communication required
 - All data stored in memory only
-

5. Non-Functional Requirements

5.1 Performance

- Application must respond to user input within 2 seconds
- Support at least 100 users and 1000 messages during a session
- Search functionality must complete within reasonable time

5.2 Reliability

- System should not crash on invalid input

- Error messages must be user-friendly
- Memory management must prevent leaks

5.3 Security

- Passwords must not be displayed when typing
- Users can only delete their own messages
- Users cannot access chats they are not part of

5.4 Usability

- Menu options must be clear and consistent
- Helpful error messages for invalid inputs
- Display current user and time information

6. Class Specifications

6.1 User Class

Data Members:

- string username: Unique identifier for the user
- string password: User's password (store as string for simplicity)
- string phoneNumber: Contact number
- string status: Online/Offline/Custom status
- string lastSeen: Timestamp of last activity

Constructor Requirements:

1. Default constructor: Initialize with empty strings and current timestamp
2. Parameterized: Set username, password, phone, and update lastSeen

Method Requirements:

Method	Input	Output	Behavior
getUsername	None	string	Return username
getPhoneNumber	None	string	Return phone number
getStatus	None	string	Return current status
getLastSeen	None	string	Return last seen timestamp
setStatus	string newStatus	void	Update status
setPhoneNumber	string phone	void	Update phone number
updateLastSeen	None	void	Set lastSeen to current time
checkPassword	string pwd	bool	Compare with stored password
changePassword	string newPwd	void	Update password

6.2 Message Class

Data Members:

- string sender: Username of sender
- string content: Message text
- string timestamp: When message was sent
- string status: sent/delivered/read
- Message* replyTo: Pointer to message being replied to

Constructor Requirements:

1. Default: Initialize with empty values and null reply
2. Parameterized: Set sender, content, current timestamp, status="sent", replyTo=nullptr

Method Requirements:

Method	Input	Output	Behavior
getContent	None	string	Return message content
getSender	None	string	Return sender username
getTimestamp	None	string	Return timestamp
getStatus	None	string	Return status
getReplyTo	None	Message*	Return pointer to replied message
setStatus	string newStatus	void	Update status
setReplyTo	Message* msg	void	Set reply reference
updateTimestamp	None	void	Set to current time
display	None	void	Print formatted message
addEmoji	string emojiCode	void	Replace code with emoji symbol

6.3 Chat Class (Base)

Data Members:

- vector<string> participants: List of usernames in chat
- vector<Message> messages: All messages in chat
- string chatName: Name of chat

Constructor Requirements:

1. Default: Empty vectors, empty name

2. Parameterized: Set participants and name

Method Requirements:

Method	Input	Output	Behavior
addMessage	const Message& msg	void	Add message to vector
deleteMessage	int index, const string& username	bool	Delete if user is sender, return success
displayChat	None	void (virtual)	Display all messages
searchMessages	string keyword	vector<Message>	Return messages containing keyword

6.4 PrivateChat Class

Data Members:

- string user1: First participant
- string user2: Second participant

Constructor Requirements:

1. Initialize with two users, set participants vector, chatName = "Chat between user1 and user2"

Method Requirements:

Method	Input	Output	Behavior
displayChat	None	void (override)	Show messages with typing indicator
showTypingIndicator	const string& username	void	Print "[username] is typing..."

6.5 GroupChat Class

Data Members:

- `vector<string> admins`: Admin usernames
- `string description`: Group description

Constructor Requirements:

1. Set participants, name, add creator to participants and admins

Method Requirements:

Method	Input	Output	Behavior
<code>addAdmin</code>	<code>string newAdmin</code>	<code>void</code>	Add to admins vector
<code>removeParticipant</code>	<code>string admin, string userToRemove</code>	<code>bool</code>	Admin removes user, return success
<code>isAdmin</code>	<code>string username</code>	<code>bool</code>	Check if user is admin
<code>isParticipant</code>	<code>string username</code>	<code>bool</code>	Check if user in participants
<code>setDescription</code>	<code>string desc</code>	<code>void</code>	Update description
<code>displayChat</code>	None	<code>void</code> (override)	Show group info and messages
<code>sendJoinRequest</code>	<code>const string& username</code>	<code>void</code>	Request to join group

6.6 WhatsApp Class

Data Members:

- `vector<User> users`: All registered users
- `vector<Chat*> chats`: All active chats
- `int currentUserIndex`: Index of logged-in user (-1 if none)

Method Requirements:

Method	Input	Output	Behavior
findUserIndex	string username	int	Linear search, return index or -1
isLoggedIn	None	bool	Check currentUserIndex != -1
getCurrentUsername	None	string	Return username of current user
signUp	None	void	Register new user, check uniqueness
login	None	void	Authenticate user
startPrivateChat	None	void	Create chat with another user
createGroup	None	void	Create group with participants
viewChats	None	void	Display all user's chats
logout	None	void	Set currentUserIndex = -1

7. Implementation Guidelines

7.1 Phase 1: User Class

Tasks:

1. Implement constructors with proper initialization
2. Create getters and setters
3. Implement updateLastSeen using <ctime> functions
4. Add password validation

7.2 Phase 2: Message Class

Tasks:

1. Implement constructors
2. Create display method with formatting
3. Implement emoji replacement
4. Add status management

Emoji Mapping:

- ":)" -> "😊"
- ":(" -> "😞"
- ":D" -> "😄"
- "<3" -> "❤️"
- ":thumbsup:" -> "👍"

7.3 Phase 3: Chat Classes

Tasks:

1. Implement Chat base class
2. Add message management
3. Create search functionality
4. Complete PrivateChat
5. Complete GroupChat with admin functions

7.4 Phase 4: WhatsApp App

Tasks:

1. Implement user management
2. Create menu system
3. Add chat creation
4. Implement view chats
5. Test all functionalities

7.5 Phase 5: Testing and Refinement

Tasks:

1. Test all edge cases
 2. Validate input handling
 3. Fix bugs
 4. Complete documentation
 5. Ensure memory management is proper
-

8. Sample Test Cases

TC1: User Registration

- Input: username="alice", password="pass123", phone="555-0101"
- Expected: User created, can login with credentials

TC2: Private Chat

- Users: alice and bob logged in
- Expected: Can create chat, send messages, see messages

TC3: Group Admin

- Creator: alice, members: bob, charlie
- Expected: Alice can add admins, remove members

TC4: Message Search

- Search keyword "hello" in chat
- Expected: Returns all messages containing "hello"

TC5: Message Reply

- Send message, reply to it
- Expected: Reply shows reference to original message